Labsheet 4 : BTCS301P-AS04

Computer Vision Lab

Khushi Choudhary CU240251564

In [ ]:
```python
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Step 1: Load Image in Grayscale
image_path = 'sample_image.png'
img = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
if img is None:
    img = np.zeros((300, 300), dtype=np.uint8)
    cv2.rectangle(img, (50, 50), (250, 250), 255, -1)
    cv2.circle(img, (150, 150), 50, 0, -1)
z    noise = np.random.normal(0, 15, img.shape).astype(np.int16)
    img = np.clip(img.astype(np.int16) + noise, 0, 255).astype(np.uint8)

# Steps 2 & 3: Compute DFT and Shift Zero-Frequency to Center
dft = np.fft.fft2(img)
dft_shift = np.fft.fftshift(dft)

# Step 4: Compute Magnitude Spectrum for Visualization
magnitude_spectrum = 20 * np.log(np.abs(dft_shift) + 1)

# Steps 5 & 6: Design Ideal Low-Pass and High-Pass Masks
rows, cols = img.shape
crow, ccol = rows // 2, cols // 2  # Center coordinates
radius = 40                        # Cutoff frequency (D0)

# Create grid of distances from center
y, x = np.ogrid[:rows, :cols]
distance_from_center = np.sqrt((x - ccol)**2 + (y - crow)**2)

# Ideal Low-Pass Filter (LPF) Mask: Pass frequencies within radius
lpf_mask = np.zeros((rows, cols), dtype=np.float32)
lpf_mask[distance_from_center <= radius] = 1.0

# Ideal High-Pass Filter (HPF) Mask: Pass frequencies outside radius
hpf_mask = 1.0 - lpf_mask

# Apply masks to shifted DFT
fshift_lpf = dft_shift * lpf_mask
fshift_hpf = dft_shift * hpf_mask

# Step 7: Inverse Shift and Inverse DFT to Reconstruct Images
f_ishift_lpf = np.fft.ifftshift(fshift_lpf)
img_lpf = np.fft.ifft2(f_ishift_lpf)
img_lpf = np.abs(img_lpf)

# High-Pass Filter Reconstruction
f_ishift_hpf = np.fft.ifftshift(fshift_hpf)
img_hpf = np.fft.ifft2(f_ishift_hpf)
img_hpf = np.abs(img_hpf)

# Step 8: Visualization and Comparison
plt.figure(figsize=(12, 10))

plt.subplot(2, 2, 1)
plt.imshow(img, cmap='gray')
plt.title('Original Image')
```

```
plt.axis('off')

plt.subplot(2, 2, 2)
plt.imshow(magnitude_spectrum, cmap='gray')
plt.title('Magnitude Spectrum (Shifted DFT)')
plt.axis('off')

plt.subplot(2, 2, 3)
plt.imshow(img_lpf, cmap='gray')
plt.title(f'Low-Pass Filtered (Cutoff = {radius})')
plt.axis('off')

plt.subplot(2, 2, 4)
plt.imshow(img_hpf, cmap='gray')
plt.title(f'High-Pass Filtered (Cutoff = {radius})')
plt.axis('off')

plt.tight_layout()
plt.show()
```

```
[ WARN:0@1.559] global loadsave.cpp:278 findDecoder imread_('sample_image.png'): can't open/read file: check file path/integrity
```

